NLU course project - Part 1A and Part 1B

Emanuele Ricca (258437)

University of Trento

emanuele.ricca@studenti.unitn.it

1. Introduction

This report investigates the optimization and evaluation of autoregressive language models on the Penn TreeBank corpus [3]. The work is divided into two main parts: improving a GPT2-style model trained from scratch and applying parameter-efficient fine-tuning to a pre-trained GPT2 model.

In the first part, I progressively improve a custom GPT2-style architecture [5] starting from the baseline configuration provided in the laboratory material. The experiments focus on scaling model capacity by modifying hidden dimensions, attention heads, network depth, and feed-forward layers. In addition, different regularization strategies are evaluated, including dropout layers [6] and weight tying [4]. Each modification is tested incrementally and evaluated using perplexity, with the goal of achieving a validation perplexity below the required threshold (PPL < 250).

In the second part, a manually implemented Low-Rank Adaptation (LoRA) framework [1] is used to fine-tune a pre-trained GPT2 model [5]. LoRA adapters are introduced into the attention projections while keeping the original model weights frozen. The objective is to compare the performance of parameter-efficient fine-tuning against the best model trained from scratch under the same evaluation metric. Generative AI tools were used only for debugging support, code verification, and document formatting.

2. Implementation Details

The experimental setup consists of two components: a GPT2-style language model trained from scratch and a manually implemented LoRA fine-tuning framework.

The custom model includes token embeddings, learnable positional embeddings, masked causal self-attention layers [7], and a linear language modeling head. Starting from the baseline configuration ($d_{\text{model}} = 20$, $n_{\text{heads}} = 1$, $num_{\text{layers}} = 1$, $ff_{\text{dim}} = 20$), the architecture is incrementally expanded up to ($d_{\text{model}} = 64$, $n_{\text{heads}} = 2$, $num_{\text{layers}} = 2$, $ff_{\text{dim}} = 128$). Four dropout mechanisms [6] are evaluated independently: embedding dropout, attention dropout, projection dropout, and feed-forward dropout. Weight tying [4] is also introduced by sharing the parameters of the token embedding layer and the final output projection. Models are trained with AdamW [2] for up to 20 epochs using a 3-epoch linear warmup schedule and early stopping with a patience of 3 epochs. Learning rates between 0.01 and 0.005 are explored during model scaling.

For Part 1.B, LoRA adapters [1] are implemented manually without using external PEFT libraries. Low-rank adaptation matrices are applied to the query, key, and value projections within each attention block [7]. Given a frozen weight matrix $W_0 \in \mathbb{R}^{d\times k}$, the adapted weight matrix is computed as

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA \tag{1}$$

where $A \in \mathbb{R}^{r\times k}$ and $B \in \mathbb{R}^{d\times r}$ are trainable low-rank matrices. Experiments use a rank of $r = 4$ and scaling factors $\alpha \in 8, 16$. All original GPT2 parameters remain frozen and only the adapter parameters are updated. Fine-tuning is performed with AdamW [2], a learning rate of 5×10^{-4}, and a 3-epoch warmup schedule.

3. Results and Evaluation

Performance is evaluated using cross-entropy loss converted to Perplexity (PPL $= \exp(\text{loss})$). Table 1 summarizes the results obtained through the incremental modifications applied to the GPT2-style model trained from scratch, while Table 2 reports the results achieved with the manually implemented LoRA adapters.

Table 1: *Incremental optimization perplexities for Part 1.A (From Scratch).*

Incremental Modification Variant	Dev PPL	Test PPL
Baseline Layout ($lr = 0.01$)	56.68	49.51
Bigger d_{model} (32, $ff_{\text{dim}} = 64$)	46.68	41.51
More Layers ($num_{\text{layers}} = 2$)	46.75	41.45
More Heads ($n_{\text{heads}} = 2$)	46.18	40.77
Embedding Dropout (0.2)	48.65	43.02
Embedding Dropout (0.05)	46.52	41.25
Attention Dropout (0.05)	46.56	N/A
Feed-Forward Dropout (0.1)	46.17	40.96
Projection Dropout (0.05)	46.91	41.57
Weight Tying ($lr = 0.01$)	54.45	48.10
WT + FF Drop 0.1 ($lr = 0.005$)	50.53	44.77
WT + FF Drop 0.1 (Scaled Up)	**43.47**	**38.81**

Table 2: *LoRA fine-tuning parameters and perplexity metrics for Part 1.B.*

Configuration Profile	Rank	Alpha	Dev PPL	Test PPL
LoRA Run 1	4	16	24.29	N/A
LoRA Run 2 (Best)	**4**	**8**	**23.24**	**20.99**

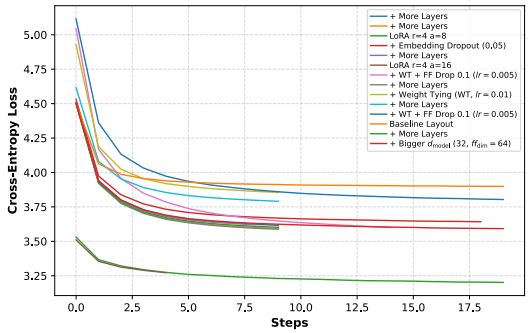
The results show clear differences between training a language model from scratch and adapting a pre-trained model. For the scratch-trained architecture, increasing d_{model} from 20 to 32 produced the largest single improvement in perplexity. Additional increases in depth and attention heads provided smaller but consistent gains. Since the Penn TreeBank corpus is relatively small, strong regularization was not necessary: larger embedding and projection dropout values increased perplexity, while a moderate feed-forward dropout of 0.1 produced the best regularization effect.

Weight tying [4] initially increased perplexity when applied to smaller model configurations. However, the reduction in parameter count made it possible to scale the model further while maintaining stable training. Combining weight tying,

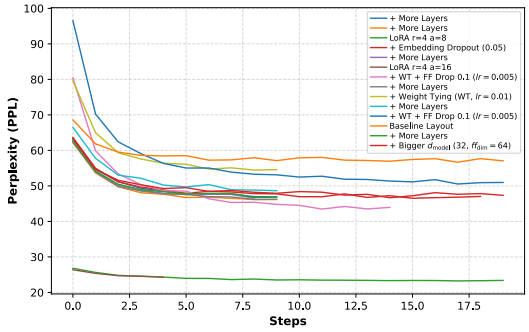
feed-forward dropout, and a larger architecture resulted in the best scratch-trained configuration, achieving a test perplexity of 38.81.

The manually implemented LoRA framework achieved substantially better results than all scratch-trained variants. By adapting a pre-trained GPT2 backbone [5] through a small number of trainable parameters, the model reached a test perplexity of 20.99 while satisfying all project requirements [1].

To further compare the optimization behaviour of the two approaches, Figure 1 presents the training loss and development perplexity trajectories collected during training.



(a) *Unified Training Loss Track*



(b) *Development Perplexity Profile*

Figure 1: *Comparison of training loss and development perplexity between the best scratch-trained models and the LoRA-based fine-tuning approach.*

As shown in Figures 1a and 1b, the models trained from scratch begin with high training loss and development perplexity values and require several epochs before stabilizing. In contrast, the LoRA models start from a much stronger initialization due to the pre-trained backbone and converge within only a few epochs.

Although each optimization step of LoRA requires passing through the full GPT2 backbone, the overall training process is considerably shorter because the model converges much more quickly than the scratch-trained alternatives. The configuration with $\alpha = 16$ was stopped early because my GPU ran out of memory.

4. References

[1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *ICLR*, 2022.

[2] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[3] Mitchell P Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.

[4] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.

[5] Alec Radford, Jeffrey Wu, Rewon Child, Dario Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

[6] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Luan Kaiser, and Illia Polosukhin. Attention is all you need. *NeurIPS*, 2017.